

Clasificación multiclase del riesgo de burnout en población Gen Z mediante redes neuronales multicapa

Patricia Rodrigo Barrio Víctor Rodríguez Albendea

Resumen

Este trabajo aborda la clasificación multiclase de la variable `Burnout_Risk` en un conjunto de datos tabular de 10.000 observaciones de población Gen Z. El problema se formuló como una tarea de aprendizaje supervisado con tres categorías de riesgo: Low, Medium y High. El flujo de trabajo incluyó una partición estratificada en entrenamiento, validación y test, estandarización z-score de las variables numéricas, codificación one-hot de las variables categóricas y ponderación de clases para compensar el fuerte desbalanceo de la categoría Low. Se compararon distintas configuraciones de perceptrón multicapa, incluyendo un baseline lineal, redes shallow, redes profundas, variantes con dropout, diferentes funciones de activación, optimizadores alternativos y una búsqueda automática de hiperparámetros mediante Keras Tuner. Debido al desbalanceo de clases, la selección del modelo final se realizó utilizando el F1 macro en validación como métrica principal, reservando el conjunto de test exclusivamente para la evaluación final. El mejor modelo fue una red profunda fully connected con arquitectura 512-256-128-64-32, activación sigmoid, optimizador Adam y dropout de 0,25. En test, este modelo alcanzó una accuracy de 0,9980 y un F1 macro de 0,9986, cometiendo únicamente dos errores de clasificación, ambos correspondientes a muestras reales de la clase High clasificadas como Medium. No obstante, el rendimiento sobre la clase Low debe interpretarse con cautela debido a su bajo soporte en test.

Palabras clave: burnout, Gen Z, clasificación multiclase, perceptrón multicapa, redes neuronales, F1 macro, desbalanceo de clases, dropout, Keras Tuner.

1. Introducción

El burnout ha adquirido un interés creciente en población joven por su asociación con agotamiento emocional, pérdida de motivación, estrés mantenido y deterioro del bienestar. En el caso de la población Gen Z, este fenómeno puede verse influido por factores como los hábitos digitales intensivos, la calidad del sueño, la presión académica o laboral, la ansiedad, la fatiga emocional y variables relacionadas con el bienestar psicológico. Desde el punto de vista del análisis de datos, este contexto puede plantearse como un problema de clasificación supervisada, donde el objetivo es asignar a cada observación un nivel de riesgo de burnout a partir de un conjunto de variables predictoras.

En este trabajo se aborda la clasificación de la variable `Burnout_Risk`, compuesta por tres categorías: Low, Medium y High. Cada muestra del dataset está descrita por variables numéricas y categóricas asociadas a hábitos, estado emocional y estilo de vida digital. Por tanto, el modelo debe aprender la relación entre dichos predictores y la clase objetivo. Al tratarse de una tarea de clasificación multiclase exclusiva, la salida de la red se define mediante tres neuronas y una activación softmax, obteniendo una probabilidad estimada para cada categoría.

Las redes neuronales multicapa o multilayer perceptrons constituyen una alternativa adecuada para este tipo de problema, ya que permiten modelar relaciones no lineales entre variables de entrada heterogéneas. A diferencia de un clasificador lineal, un perceptrón multicapa incorpora capas ocultas con funciones de activación no lineales, lo que permite construir fronteras de decisión más flexibles [1].

El problema presenta, además, un desbalanceo muy acusado entre clases. En concreto, la clase Low está representada por un número de observaciones muy inferior al de las clases Medium y High. En estas condiciones, la accuracy puede resultar insuficiente como métrica principal, ya que un modelo podría obtener un rendimiento global elevado favoreciendo las clases mayoritarias.

Por este motivo, se emplearon pesos de clase durante el entrenamiento y se utilizó el F1 macro como métrica principal de comparación, al promediar el rendimiento de las tres clases dando el mismo peso a cada una.

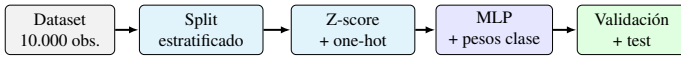
El objetivo del trabajo fue comparar distintas configuraciones de redes neuronales multicapa para clasificar el riesgo de burnout en población Gen Z, analizando el efecto de la profundidad, la función de activación, el optimizador, el learning rate, la regularización mediante dropout y la búsqueda automática de hiperparámetros con Keras Tuner. Todos los modelos se entrenaron bajo un protocolo común y se seleccionaron utilizando el F1 macro en validación, métrica más adecuada que la accuracy en presencia de desbalanceo entre clases.

2. Materiales y métodos

2.1. Dataset, partición y preprocesado

El estudio se llevó a cabo sobre un conjunto de datos tabular compuesto por 10.000 observaciones de población Gen Z y 22 variables, de las cuales una correspondía a la variable objetivo, `Burnout_Risk` [2]. Tras excluir dicha variable, el modelo se entrenó con 21 predictores asociados a hábitos digitales, estado emocional, bienestar, calidad del sueño y características personales. El problema se formuló como una tarea de clasificación supervisada multiclase con tres categorías mutuamente excluyentes: Low, Medium y High.

Del total de predictores, 17 fueron tratados como variables numéricas y 4 como variables categóricas: `Gender`, `Country`, `Working Status` y `Content Type Preference`. Las variables numéricas se transformaron mediante estandarización z-score, mientras que las categóricas se codificaron mediante one-hot encoding. Ambos transformadores se ajustaron exclusivamente sobre el conjunto de entrenamiento y posteriormente se aplicaron a validación y test, evitando así fuga de información.



El preprocesado se ajusta solo en entrenamiento; la selección se realiza en validación.

Figura 1: Esquema general del flujo de trabajo seguido: partición estratificada, preprocesado, entrenamiento de modelos, selección mediante F1 macro en validación y evaluación final sobre test.

Tras este preprocesado, la dimensión final de entrada de la red fue de 34 características. La partición de datos se realizó de forma estratificada, manteniendo aproximadamente la proporción de clases en los subconjuntos de entrenamiento, validación y test. El reparto final fue de 7999 muestras para entrenamiento, 1001 para validación y 1000 para test.

Tabla 1: Resumen del conjunto de datos, partición y preprocesado.

Característica	Valor
Observaciones totales	10.000
Variables totales	22
Predictores usados	21
Variables numéricas / categóricas	17 / 4
Dimensión final tras el preprocesado	34
Partición train / val / test	7999 / 1001 / 1000
Distribución global de la variable objetivo	Low 64; Medium 4548; High 5388
Distribución en test	Low 6; Medium 455; High 539
Pesos de clase Low / Medium / High	52,28 / 0,73 / 0,62

La estratificación fue especialmente importante debido al fuerte desbalanceo de la variable objetivo, ya que la clase Low únicamente contiene 64 observaciones en todo el dataset, frente a 4548 de la clase Medium y 5388 de la clase High. Este desbalanceo también se refleja en la distribución de pesos utilizada durante el entrenamiento, donde la clase Low recibe un peso muy superior al resto.

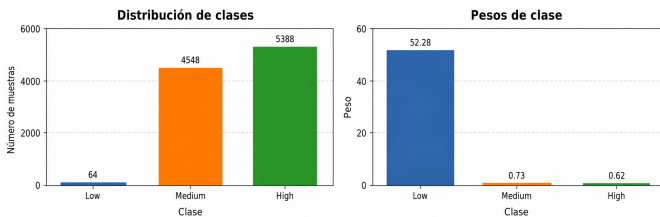


Figura 2: Distribución global de la variable Burnout_Risk y pesos de clase utilizados durante el entrenamiento. El elevado peso asignado a Low refleja su baja representación en el conjunto de entrenamiento.

Para reducir el efecto de este desbalanceo durante el entrenamiento, se aplicaron pesos de clase inversamente proporcionales a la frecuencia de cada categoría en el conjunto de entrenamiento. Estos pesos se calcularon como:

$$w_c = \frac{N_{\text{train}}}{K \cdot N_{c,\text{train}}}, \quad (1)$$

donde N_{train} es el número total de muestras de entrenamiento, K es el número de clases y $N_{c,\text{train}}$ es el número de muestras de la clase c en entrenamiento. De esta forma, los errores cometidos sobre la clase minoritaria Low tienen mayor contribución durante la actualización de los pesos de la red. Los pesos obtenidos fueron 52,28 para Low, 0,73 para Medium y 0,62 para High.

2.2. Arquitecturas comparadas

Se evaluaron distintas configuraciones de perceptrón multicapa con el objetivo de analizar el efecto de la complejidad de la arquitectura, la función de activación, el optimizador, el learning rate y la regularización mediante dropout. Todas las configuraciones utilizaron la misma entrada preprocesada de 34 características, lo que permite atribuir las diferencias de rendimiento a las decisiones de modelado y no al tratamiento de los datos. La Fig. 3 resume el esquema experimental seguido.

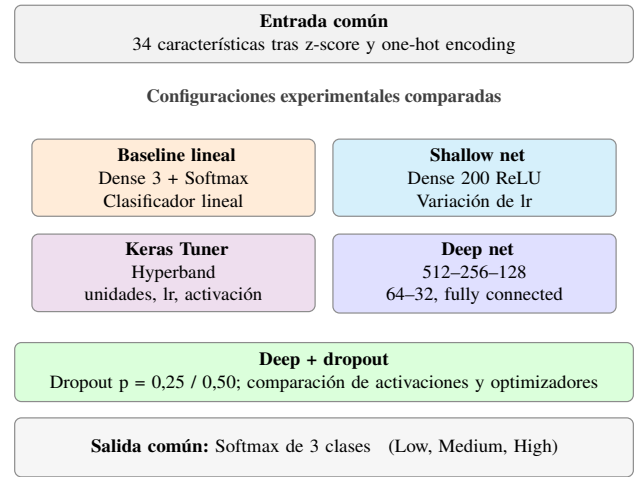


Figura 3: Resumen de las configuraciones evaluadas. Todas las alternativas parten de la misma entrada preprocesada y se comparan variando la complejidad de la arquitectura, la tasa de aprendizaje, la activación, la regularización y el optimizador.

En primer lugar, se entrenó un modelo lineal baseline, implementado como una única capa densa de salida con tres neuronas y activación softmax. Este modelo equivale a un clasificador lineal multiclase y se empleó como referencia mínima de rendimiento, ya que permite comprobar si las variables de entrada pueden separarse mediante una frontera de decisión simple.

Posteriormente, se evaluaron redes shallow con una única capa oculta de 200 neuronas y activación ReLU, modificando la tasa de aprendizaje para estudiar su influencia en la convergencia. Este bloque permitió analizar si la introducción de una primera transformación no lineal era suficiente para mejorar el rendimiento respecto al baseline lineal.

A continuación, se incorporó una búsqueda automática de hiperparámetros mediante Keras Tuner con estrategia Hyperband [4]. Esta búsqueda se utilizó como complemento a la comparación manual, no como sustituto de ella. Mientras que las configuraciones manuales permiten interpretar de forma controlada el efecto de cada decisión de diseño, Keras Tuner explora de manera automática combinaciones dentro de un espacio acotado de hiperparámetros. En este caso se consideraron distintas unidades ocultas, funciones de activación, tasas de aprendizaje y optimizadores.

Finalmente, se evaluaron redes profundas fully connected con cinco capas ocultas de tamaños 512-256-128-64-32. Sobre esta arquitectura se analizaron diferentes variantes: sin dropout, con dropout de 0,25 y 0,50, con distintas funciones de activación y con distintos optimizadores. Esta parte permitió estudiar si una arquitectura más profunda mejoraba el rendimiento respecto al baseline lineal, las redes shallow y la configuración obtenida mediante Keras Tuner.

La Tabla 2 resume los bloques experimentales comparados.

Tabla 2: Configuraciones experimentales evaluadas.

Bloque	Configuración principal
Baseline lineal	Capa densa única, 3 salidas softmax, SGD
Shallow network	1 capa oculta de 200 neuronas ReLU; distintos learning rates
Keras Tuner	Hyperband sobre unidades, learning rate, activación y optimizador
Deep network	Arquitectura 512-256-128-64-32; ReLU; Adam
Deep + dropout	Misma arquitectura profunda con dropout $p = 0,25$ y $p = 0,50$
Comparación activaciones	ReLU, tanh, sigmoid y ELU sobre la red profunda regularizada
Comparación optimizadores	Adam, SGD y RMSprop sobre la red profunda regularizada

3. Entrenamiento y evaluación

Para garantizar una comparación homogénea, todos los modelos se entrenaron bajo un protocolo experimental común. En todos los casos se utilizó la misma partición *train-validation-test*, el mismo preprocesado de entrada, un tamaño de batch de 128 muestras y un máximo de 50 épocas. El número efectivo de épocas pudo variar entre configuraciones debido al uso de *early stopping*, de forma que cada modelo se detuvo cuando dejó de mejorar su rendimiento en validación.

La función de pérdida utilizada fue la entropía cruzada categórica ponderada, incorporando los pesos de clase calculados a partir del conjunto de entrenamiento:

$$\mathcal{L}_i = - \sum_{c=1}^C w_c y_{i,c} \log(\hat{y}_{i,c}), \quad (2)$$

donde w_c es el peso asignado a la clase c , $y_{i,c}$ representa la etiqueta real en formato one-hot y $\hat{y}_{i,c}$ es la probabilidad estimada por la red para dicha clase. Esta ponderación modifica la contribución de cada muestra a la pérdida total, penalizando con mayor intensidad los errores cometidos sobre las clases menos representadas, especialmente Low.

Para reducir el riesgo de sobreajuste, se empleó *early stopping* monitorizando el F1 macro en validación y restaurando los mejores pesos encontrados durante el entrenamiento. Además, se aplicó *ReduceLROnPlateau*, reduciendo la tasa de aprendizaje cuando el rendimiento en validación dejaba de mejorar. De esta forma, el entrenamiento combinó un criterio de parada basado en generalización con un ajuste progresivo de la tasa de aprendizaje.

Dado el fuerte desbalanceo entre clases, la accuracy no se utilizó como criterio principal de selección del modelo. En su lugar, se empleó el F1 macro, ya que calcula el promedio no ponderado del F1-score de cada clase y, por tanto, otorga la misma importancia a Low, Medium y High independientemente de su frecuencia:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (3)$$

El modelo final se seleccionó exclusivamente a partir del conjunto de validación, escogiendo la configuración con mayor `best_val_macro_f1`. El conjunto de test se reservó únicamente para la evaluación final, evitando utilizarlo durante la selección de hiperparámetros. Sobre este conjunto se calcularon accuracy, precision macro, recall macro, F1 macro, métricas por clase y matriz de confusión. Este diseño experimental evita que el conjunto de test influya en la selección del modelo, permitiendo utilizarlo como una estimación final del rendimiento sobre datos no empleados durante el ajuste.

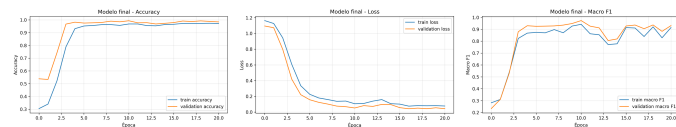


Figura 4: Curvas de aprendizaje del modelo final seleccionado: accuracy, pérdida y F1 macro en entrenamiento y validación.

Las curvas de aprendizaje muestran que el modelo converge de forma rápida durante las primeras épocas, con una reducción pronunciada de la pérdida y un incremento simultáneo de la accuracy y del F1 macro. A partir de las primeras iteraciones, las curvas de entrenamiento y validación se mantienen próximas, lo que sugiere una buena capacidad de generalización y ausencia de un sobreajuste marcado. La ligera oscilación del F1 macro en validación es esperable en un problema desbalanceado, especialmente por el bajo número de muestras de la clase Low.

4. Resultados

4.1. Comparación de configuraciones

La Tabla 3 resume el rendimiento de las configuraciones evaluadas, ordenadas según el F1 macro en validación. Además de las métricas globales, se incluye el F1-score de la clase Low, el número de épocas entrenadas y los parámetros entrenables, con el fin de comparar rendimiento, comportamiento en la clase minoritaria y complejidad del modelo.

Tabla 3: Comparación de configuraciones. La tabla se ordena por F1 macro en validación e incluye los parámetros entrenables como medida de complejidad.

Modelo	Val F1	F1 test	F1 Low	Ép.	Parám.
Deep sigmoid + Adam + drop. 0,25	0,9734	0,9986	1,000	21	192,6k
Keras Tuner reentrenado	0,9700	0,9496	0,857	17	4,9k
Deep ReLU + RMSprop + drop. 0,25	0,9522	0,9966	1,000	39	192,6k
Deep tanh + Adam + drop. 0,25	0,9468	0,9706	0,923	26	192,6k
Deep ReLU + Adam sin dropout	0,9440	0,9455	0,857	22	192,6k
Deep ELU + Adam + drop. 0,25	0,9360	0,9953	1,000	24	192,6k
Deep ReLU + Adam + drop. 0,25	0,9347	0,9720	0,923	35	192,6k
Deep ReLU + Adam + drop. 0,50	0,8788	0,8590	0,600	19	192,6k
Shallow ReLU + SGD, lr = 0,01	0,8748	0,8416	0,571	50	7,6k
Baseline lineal	0,7025	0,6669	0,179	50	0,1k
Shallow ReLU + SGD, lr = 0,001	0,6888	0,6631	0,194	50	7,6k
Deep ReLU + SGD + drop. 0,25	0,3179	0,2608	0,133	12	192,6k

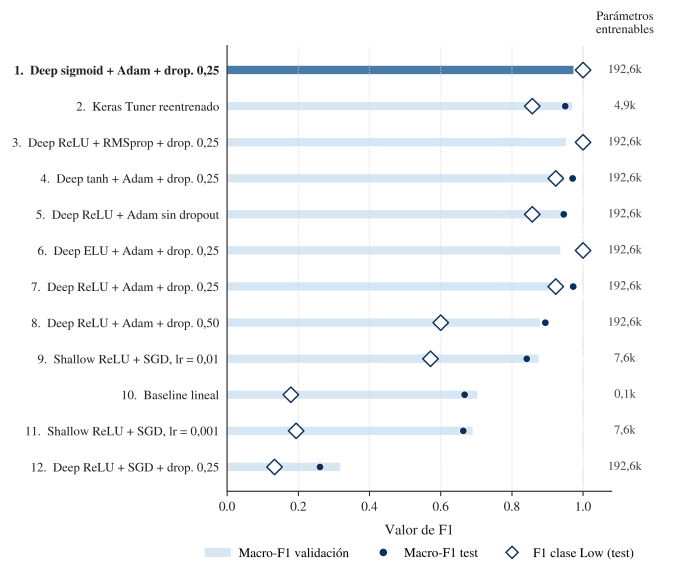


Figura 5: Resumen visual de las configuraciones evaluadas: F1 macro en validación, F1 macro en test, F1-score de la clase Low y parámetros entrenables.

A partir de estos resultados se observan tres patrones principales. En primer lugar, las arquitecturas profundas concentran los mejores rendimientos, aunque la profundidad por sí sola no garantiza una buena clasificación. La configuración profunda entrenada con SGD obtuvo el peor resultado pese a tener el mismo orden de parámetros que el resto de redes profundas, lo que indica que el optimizador y la dinámica de entrenamiento fueron factores determinantes.

En segundo lugar, se aprecia un compromiso entre capacidad del modelo y generalización. El baseline lineal, con solo 105 parámetros, y las redes shallow, con aproximadamente 7,6k parámetros, quedaron claramente por debajo de las arquitecturas profundas, especialmente en la clase Low. Por su parte, la configuración obtenida mediante Keras Tuner fue mucho más compacta, con aproximadamente 4,9k parámetros, y alcanzó un F1 macro de validación muy competitivo. Sin embargo, su rendimiento en test, especialmente sobre Low, fue inferior al del modelo finalmente seleccionado.

El mejor equilibrio se obtuvo con la red profunda con activación sigmoid, optimizador Adam y dropout $p = 0,25$. Esta configuración alcanzó el mayor F1 macro en validación, 0,9734, y mantuvo un rendimiento muy elevado en test, con un F1 macro de 0,9986. Aunque la red ReLU + RMSprop + dropout $p = 0,25$ también obtuvo resultados muy competitivos en test, no superó al modelo seleccionado bajo el criterio definido previamente en validación.

4.2. Análisis del modelo final

El modelo final fue `deep_sigmoid_adam_dropout025`, una red fully connected profunda con capas ocultas 512-256-128-64-32, activación sigmoid, optimizador Adam y dropout $p = 0,25$. Esta arquitectura cuenta con 192.579 parámetros entrenables.

En el conjunto de test, el modelo alcanzó una accuracy de 0,9980, una precision macro de 0,9985, un recall macro de 0,9988 y un F1 macro de 0,9986. A nivel de clase, Low obtuvo precision, recall y F1-score iguales a 1,000, aunque únicamente con 6 muestras en test. Por ello, este resultado debe interpretarse con cautela.

La matriz de confusión del modelo final se muestra en la Fig. 6. El modelo cometió únicamente dos errores de clasificación en las 1000 observaciones del conjunto de test, ambos correspondientes a muestras reales de la clase High clasificadas como Medium. Las clases Low y Medium fueron clasificadas correctamente en todas sus muestras de test.

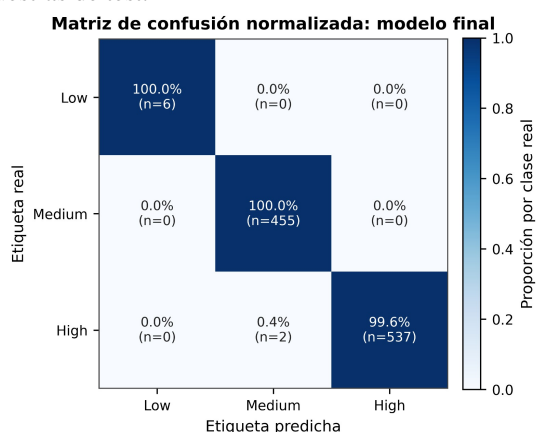


Figura 6: Matriz de confusión en test del modelo final seleccionado por F1 macro en validación.

5. Discusión y conclusiones

Los resultados obtenidos muestran que las redes neuronales multicapa fueron capaces de clasificar la variable `Burnout_Risk` con un rendimiento muy elevado en el conjunto de datos analizado. Frente al baseline lineal, las arquitecturas con capas ocultas mejoraron claramente el F1 macro y el comportamiento sobre la clase minoritaria, lo que sugiere que la relación entre los predictores y el riesgo de burnout no queda completamente capturada por una frontera lineal simple.

La comparación entre configuraciones evidencia que la arquitectura y los hiperparámetros tuvieron un impacto relevante. Las redes shallow mejoraron al baseline cuando se utilizó una tasa de aprendizaje adecuada, pero no alcanzaron el rendimiento de las arquitecturas profundas. Asimismo, los modelos entrenados con Adam o RMSprop fueron claramente más competitivos que los basados en SGD, probablemente por su capacidad para adaptar la tasa de aprendizaje durante la optimización.

El uso de dropout también influyó en la generalización. Con dropout $p = 0,25$ se obtuvo el mejor equilibrio entre capacidad de ajuste y regularización, mientras que aumentar la probabilidad a $p = 0,50$ redujo el rendimiento, especialmente en la clase Low.

Este resultado indica que la regularización fue útil, pero que una intensidad excesiva puede limitar el aprendizaje de patrones asociados a clases con muy pocas observaciones.

La búsqueda automática mediante Keras Tuner proporcionó una configuración compacta y competitiva, con un F1 macro de validación de 0,9700. Sin embargo, el modelo finalmente seleccionado fue la red profunda con activación sigmoid, optimizador Adam y dropout $p = 0,25$, al obtener el mayor F1 macro en validación. Esta comparación permite interpretar Keras Tuner como una herramienta complementaria a la exploración manual: facilita encontrar combinaciones prometedoras, mientras que la comparación controlada permite analizar el efecto individual de la profundidad, la activación, el optimizador y la regularización.

Desde un punto de vista metodológico, la selección del modelo se basó únicamente en el rendimiento de validación, manteniendo el conjunto de test como evaluación final independiente. Esta separación es especialmente importante en este estudio, ya que los valores de test son muy elevados y podrían inducir a una interpretación excesivamente optimista si no se contextualizan junto con el desbalanceo de clases y el bajo número de muestras Low.

La principal limitación del estudio es el fuerte desbalanceo de la variable objetivo. Aunque se aplicaron pesos de clase y se utilizó F1 macro como métrica principal, el bajo soporte de Low limita la robustez de las conclusiones específicas sobre esta categoría. Además, al tratarse de un dataset tabular público, algunas variables predictoras podrían estar próximas semánticamente a la propia definición de burnout, lo que facilitaría la separación entre clases y ayudaría a explicar los valores tan elevados obtenidos.

Como líneas futuras, sería recomendable repetir el experimento con varias semillas o mediante validación cruzada estratificada, evaluar el modelo en un conjunto externo y estudiar la importancia de las variables de entrada. También podría analizarse el rendimiento eliminando variables demasiado relacionadas con la definición directa de `Burnout_Risk`, planteando así un escenario predictivo más exigente.

En conclusión, el trabajo muestra que un perceptrón multicapa adecuadamente ajustado permite clasificar el riesgo de burnout en población Gen Z con un rendimiento muy alto. El mejor modelo alcanzó una accuracy de 0,9980 y un F1 macro de 0,9986 en test, cometiendo únicamente dos errores de clasificación. Aun así, estos resultados deben interpretarse como rendimiento predictivo dentro del dataset analizado, y no como una validación clínica externa.

Referencias

- [1] J. Marín-Morales, *Fundamentos de las redes neuronales y deep learning*. Material docente de la asignatura Simulación y Redes Neuronales, Universitat Politècnica de València, 2026.
- [2] H. Ansari, *Gen Z Mental Wellness & Digital Lifestyle Patterns*. Kaggle, 2025. Dataset en línea.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [4] F. Chollet et al., *Keras*. GitHub, 2015.
- [5] D. P. Kingma y J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.